

Circles — Development Plan & Methodology Selection

Assignment 2

3 April 2026

SUBMITTED TO

Software Development Case Study · CSE-2507M

SUBMITTED BY

Alexandr Tyulkov

Contents

1. Methodology Selection	1
1.1. Scoring Matrix	1
1.2. Justification	1
2. Project Timeline & Milestones	2
2.1. Gantt Chart	3
3. Requirements Traceability	3
3.1. Functional Requirements	3
3.2. Non-Functional Requirements	4
4. Risk Register	6
4.1. Task Dependencies	8
5. AI Tools Usage	8

1. Methodology Selection

1.1. Scoring Matrix

Score: 0 = not a fit, 1 = partial, 2 = strong fit. Scored against Circles specifically – one developer, 10 weeks, and a feature list that changed a lot during development.

Criterion	Waterfall	Scrum	Kanban	SAFe
Requirements known up-front	0	2	2	1
Team size fit (solo = 1)	0	1	2	0
Stakeholder involvement	1	2	1	0
Tolerance for change	0	2	2	2
Timeline flexibility	0	2	2	1
Incremental delivery	0	2	2	0
Best suited for this context	0	2	1	0
Total / 14	1	13	12	4

1.2. Justification

Scrum scored 13/14 and was selected. One-week sprints are used across the 10-week semester.

The feature list for Circles wasn't settled at the start. Authentication and basic stats were planned first. Things like AI personality reports, playlist generation, and a ZIP importer came later, as it became clear they were feasible. Scrum works well here because the backlog can be updated between sprints – there's no need to plan everything upfront. Kanban scored 12/14 and was close, but it has no fixed cadence. For a project with a hard deadline, having a sprint boundary every week is more useful than open-ended flow.

Stakeholder involvement scored 2 because each sprint ends with a self-review against the sprint goal. Even solo, this creates a regular checkpoint.

Scrum assumes 3 to 9 people, so team size scored 1. Daily stand-ups don't make sense for one developer. In practice: a short task note each morning and a 5-minute checklist at sprint end.

Trade-off 1 – ceremonies take time. Planning and reviewing every sprint adds up. In practice it's kept to 10 minutes – picking 2 or 3 tasks from the backlog, nothing more.

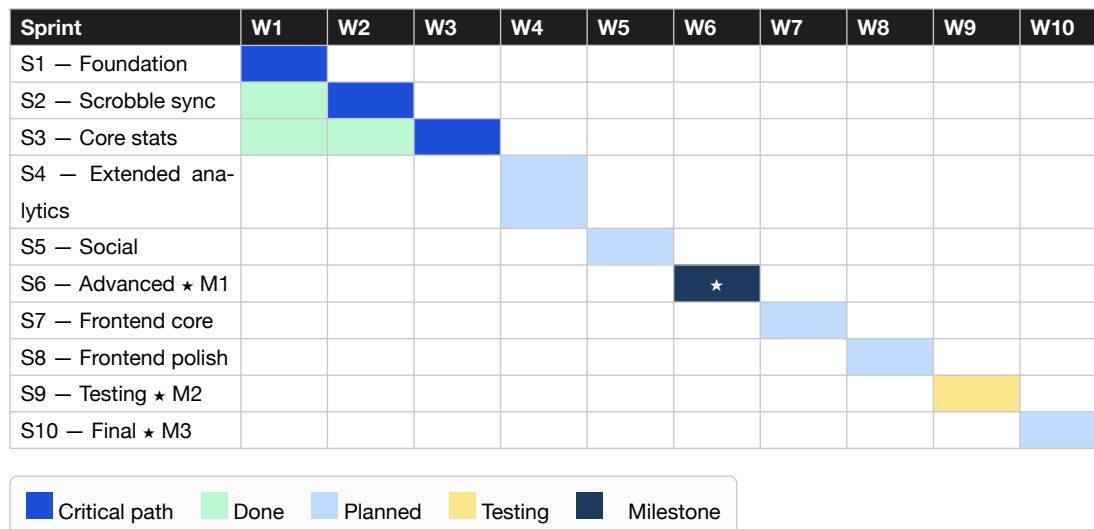
Trade-off 2 – one week is short. A task that runs 2 days over leaves almost no room to recover. Sprint goals are kept small on purpose. If a feature isn't finished, it moves to the next sprint rather than blocking everything else.

2. Project Timeline & Milestones

Sprint	Week	Dates	Goal	Status
S1	W1	17–23 Mar	DB schema, Spotify OAuth, auth setup	✓ Done
S2	W2	24–30 Mar	Scrobble sync background worker, token-refresh cron	✓ Done
S3	W3	31 Mar–6 Apr	Top artists/albums/tracks endpoints with time-range params	In Progress
S4	W4	7–13 Apr	Time machine, audio features, extended stats	Not Started
S5	W5	14–20 Apr	Follow/unfollow, leaderboard, music-match, public profiles	Not Started
S6	W6	21–27 Apr	Playlists + auto-cron, AI insights, ZIP import – M1	Not Started
S7	W7	28 Apr–4 May	Frontend core — stats dashboards, charts, main routes	Not Started
S8	W8	5–11 May	Frontend polish — social/AI/playlist UI, responsive, WCAG	Not Started
S9	W9	12–18 May	Unit tests, E2E, NFR validation, bug fixes – M2	Not Started
S10	W10	19–25 May	Deployment, docs, final submission – M3	Not Started

Milestone	Date	Criteria
M1 Prototype Demo	27 Apr	Stats and social features return real data from the API; at least one page in the UI shows live data
M2 Testing Complete	18 May	All tests pass; no critical bugs open; performance and auth checks verified
M3 Final Submission	25 May	App is deployed and accessible; all documents submitted

2.1. Gantt Chart



Critical path is S1 to S4 (dark blue). S1 is the most important sprint – the database schema it creates is what every later feature reads from. If S1 slips, everything else slips too.

3. Requirements Traceability

All requirements from Assignment 1 are listed below with the sprint they belong to and current status. Tasks are scoped to fit within 1 to 3 days of work.

3.1. Functional Requirements

ID	Summary	Tasks	Sprint	Status
FR-01	Spotify OAuth + auto scrobble sync	Set up auth flow with Spotify; store listening history automatically in the background; schedule token refresh	S1–S2	✓ Done
FR-02	Top artists/albums/tracks, time ranges	Build stats endpoints with configurable time windows; add genre and audio-feature summaries	S3	In Prog
FR-03	Follow/unfollow + activity feed	Social graph storage; follow/unfollow actions; feed showing friends' recent listening	S5	–
FR-04	AI personality reports	Build listening-history context; call AI API with three prompt types (roast, taste profile, scene); cache results weekly	S6	–

ID	Summary	Tasks	Sprint	Status
FR-05	Playlist CRUD + auto weekly playlist	Create/read/update/delete playlists; auto-generate a weekly playlist from recent listening on a schedule	S6	–
FR-06	Global leaderboard	Aggregate total scrobbles per user; expose weekly and all-time rankings with short-lived cache	S5	–
FR-07	ZIP historical data import	Accept Spotify data export ZIP; unpack and bulk-insert historical tracks into the database via background job	S6	–
FR-08	Public user profiles	Add username, bio, and visibility setting to user accounts; serve a public profile page per username	S5	–
FR-09	Music-match score	Score overlap between two users' top artists and tracks; expose as a single match percentage	S5	–
FR-10	Time-machine view	Query listening history for a specific past date and return tracks played that day	S4	–

3.2. Non-Functional Requirements

ID	Summary	Tasks	Sprint	Status
NFR-P1	API p95 $\leq$ 200 ms	Add database indexes on user + date columns; profile slow endpoints; add response-time logging	S1/S9	S1 ✓
NFR-P2	Sync $\leq$ 30 s/user	Process track metadata in batches; measure sync duration in background worker	S2/S9	S2 ✓
NFR-P3	Leaderboard $\leq$ 500 ms	Cache leaderboard result for 5 minutes; validate latency under load	S5/S9	–

ID	Summary	Tasks	Sprint	Status
NFR-P4	Frontend FCP ≤ 2 s	Audit bundle size; enable server-side rendering; lazy-load chart components	S8/S9	–
NFR-U1	Responsive ≥ 375 px	Apply responsive styles to all layouts; verify on narrow viewport	S8	–
NFR-U2	WCAG 2.1 AA contrast	Run automated accessibility audit; fix any contrast failures	S8	–
NFR-U3	Onboarding ≤ 60 s	Streamline OAuth redirect; show loading skeleton while data fetches	S1/S8	S1 ✓
NFR-U4	Human-readable errors	Global error handler mapping failures to plain-language messages; friendly fallback states	S3/S8	Partial
NFR-S1	HTTPS / TLS 1.2+	Configure TLS on production host; redirect plain HTTP; run SSL audit	S10	–
NFR-S2	Tokens encrypted at rest	Verify auth library stores tokens encrypted; confirm tokens never appear in API responses	S1/S9	S1 ✓
NFR-S3	401 on unauthenticated	Require valid session on all protected routes; write test asserting 401 for missing session	S1/S9	S1 ✓
NFR-S4	Private data inaccessible	Apply visibility check on all social endpoints; test both public and private account scenarios	S5/S9	–
NFR-R1	$\geq 99.5\%$ uptime	Add health-check endpoint; set up uptime monitoring; write incident runbook	S10	–

ID	Summary	Tasks	Sprint	Status
NFR-R2	Token refresh $\leq$ 50 min	Confirm background refresh job fires every 50 minutes reliably	S2/S9	S2 ✓
NFR-R3	Auto-retry x3, back-off	Configure retries with back-off on all background workflows; test with simulated failures	S2/S6/S9	S2 ✓
NFR-R4	Graceful Spotify 429	Detect rate-limit responses; pause and retry automatically; never expose the error to users	S2/S9	S2 ✓
NFR-SC1	No degradation at 10 M rows	Profile aggregation queries against a large synthetic dataset; add covering indexes where needed	S1/S9	S1 ✓
NFR-SC2	Horizontal worker scaling	Verify background jobs are deduplicated when multiple worker instances run; document how to scale	S2/S9	S2 ✓
NFR-SC3	Stateless API server	Confirm all session state lives in the database, not in-process; verify load-balancer compatibility	S1/S9	S1 ✓

4. Risk Register

Risk (category)	Prob.	Impact	Priority	Mitigation	Owner & By
Spotify changes or removes an API endpoint (technical)	M	H	P1	Keep Spotify calls behind a wrapper layer so changes are easy to fix in one place; handle missing data gracefully	Dev – S3
Stats queries become too slow with	L	H	P2	Test queries against a large dataset before	Dev – S9

Risk (category)	Prob.	Impact	Priority	Mitigation	Owner & By
real user data (technical)				launch; add indexes; cache the leaderboard result	
Getting sick or stuck for several days (resource)	M	H	P1	Push code to remote every day so no work is lost; keep sprint goals small so one bad day does not break the whole sprint	Dev – ongoing
Adding too many features and running out of time (timeline)	H	M	P1	Stop adding features after Sprint 6; anything new goes into a backlog for later	Dev – 27 Apr
Not enough time left for testing (timeline)	M	M	P2	Sprint 9 is reserved only for testing; write tests for each feature as it is finished, not all at the end	Dev – S5
Personality report service gets expensive (resource)	L	M	P3	Store the generated report and reuse it for a week; only call the service again when the week is over	Dev – S6
Bus factor is 1 (resource)	H	H	P1	One developer means one point of failure. If that person is unavailable for any reason, the project stops. Com-	Dev – ongoing

Risk (category)	Prob.	Impact	Priority	Mitigation	Owner & By
				mit messages are kept descriptive, non-obvious decisions are commented in code, and credentials are stored in a password manager so the project can be picked up after a break	

4.1. Task Dependencies

Upstream	Downstream	Why
S1: DB schema	S3: Analytics	All stat queries read from history; schema must be final first
S2: Scrobble sync	S4: Time machine and extended stats	Time-series analytics need real scrobble data in the database
S5: Social graph	S5: Activity feed and music match	Follow relationships must exist before feed or match queries work
S6: All backend features	S7: Frontend integration	Building UI against incomplete endpoints wastes effort
S8: Frontend complete	S9: E2E testing	End-to-end tests require both frontend and backend to be stable
S9: All tests pass	S10: Final submission	Submission requires a working, tested system

5. AI Tools Usage

Tool	Used for	Kept / changed / rejected
Claude (claude-sonnet-4-6)	Helped structure the scoring matrix and sug-	Scores were set based on the actual project

Tool	Used for	Kept / changed / rejected
	gested an initial list of risks to consider.	context. Risk descriptions were rewritten to reflect realistic solo-developer scenarios. Several suggestions that did not apply were removed.